

Advanced Software Development

Containers



Simon H. Muchinenyika

Containers

Table of Contents

- Introduction
- Containers: Docker, LXC
- Container Orchestration
 - GKE/EKS/AKS, AWS ECS/Fargate, Docker Swarm, Kubernetes, OpenShift
- Underlying Technologies
- namespaces
- Cgroups

Containers

Objectives

- At the end of this class, students should have developed skills to start developing Kubernetes and containerized applications.
- Some practical exercises covered include:
 - creating a Dockerfile for a sample application.
 - building an image.
 - running the image locally in a container.
 - change container behaviour.
 - push the image to Artifact Registry.

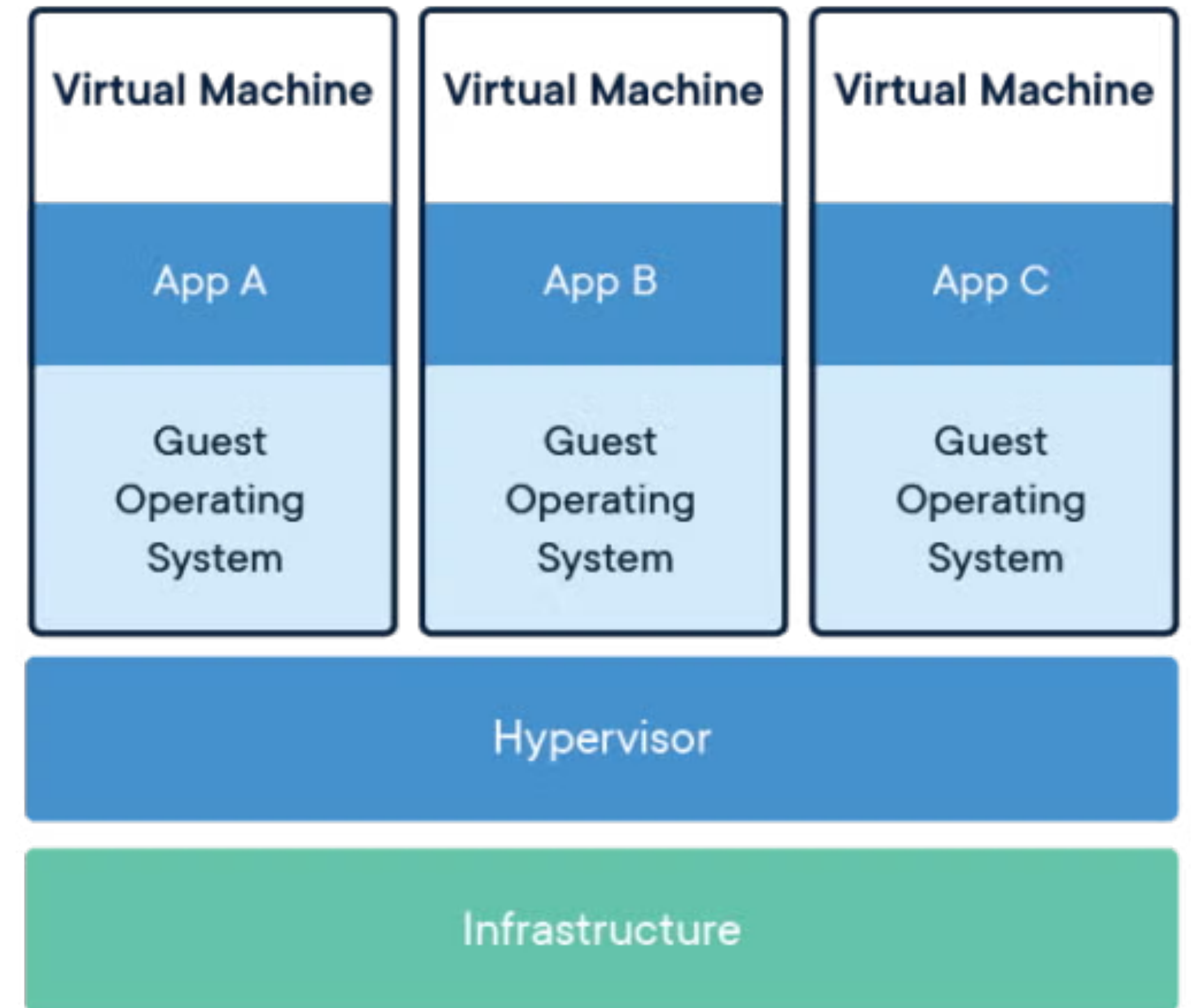
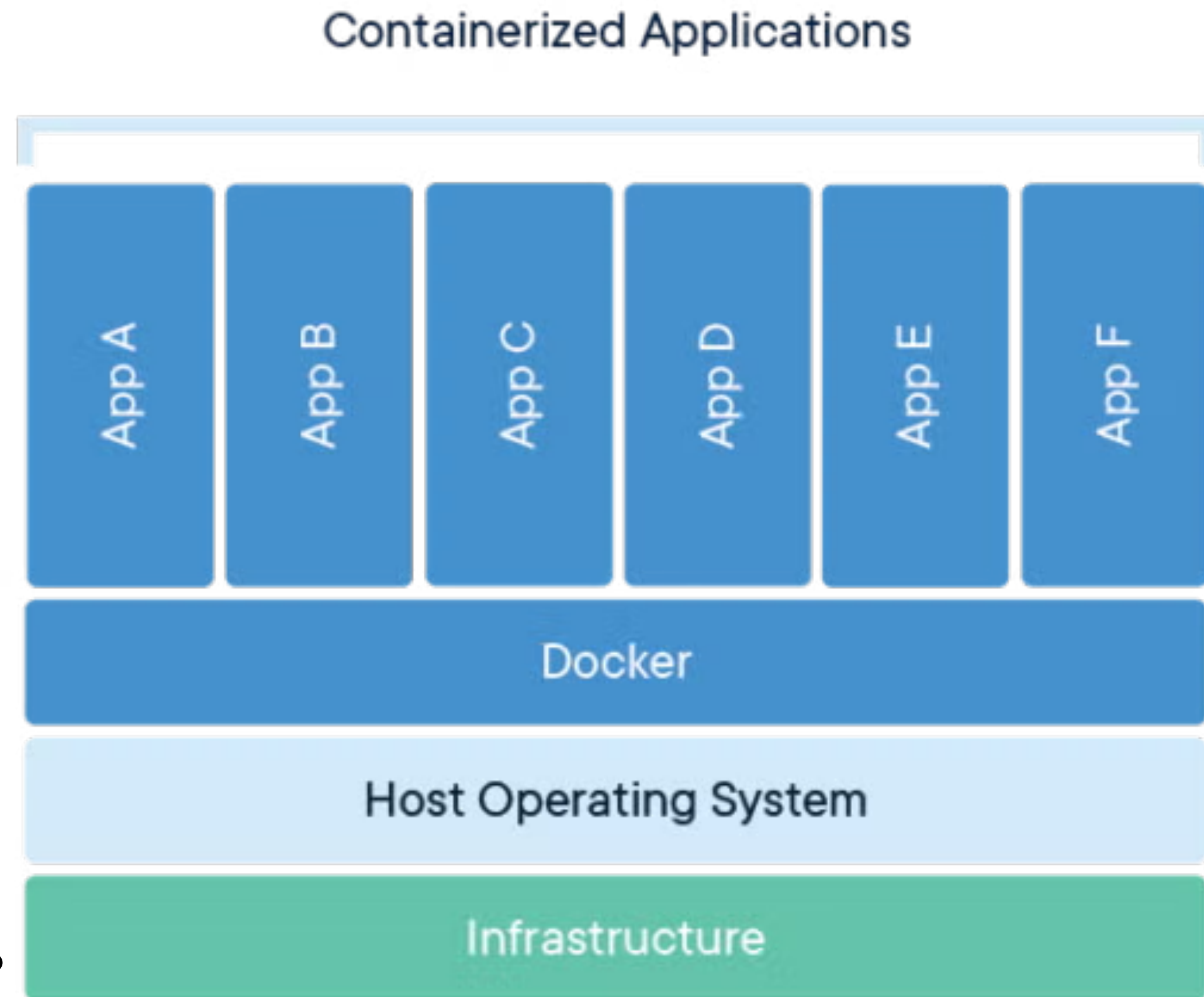
Containers

Introduction

- loosely isolated environments in which applications run independently of their hosts.
- shared containers guarantee the same results regardless of the recipient.
- a container becomes the unit for testing and distributing your application.
- application is deployed into production as a container or an orchestrated service.

Containers

Containers Vs Virtual Machines



Containers

Virtualisation

- The **hypervisor** sits between the hardware and VMs, managing allocation of resources.
- Types of hypervisors:
 - Type 1 (bare-metal) runs directly on the hardware, e.g., VMware ESXi.
 - Type 2 (hosted) runs on the existing OS, e.g., VirtualBox.
- Before virtualisation, running one application required a dedicated physical server.
- This was inefficient since CPU & memory resources were underutilised.

Containers

Docker

- an open platform for developing, shipping, & running applications.
- used to separate applications from infrastructure & treats infrastructure like a managed application.
- makes it possible to ship code faster, test faster, deploy faster, & shorten the cycle between writing code and running code.
- makes it possible by combining kernel containerisation features with workflows & tooling that help in managing & deploying applications.
- Docker containers can be directly used in Kubernetes, allowing them to run in the Kubernetes engine with ease.

Containers

Containerisation

- an application may be packaged into a container using a [Dockerfile](#).
- the Dockerfile is a script that instructs the daemon on how to assemble the container image.
- its always starts with the [FROM](#) instruction, which specifies the base image from which the building is done.
- a [container image](#) is a standardised package that includes all files, binaries, libraries, & configurations to run a container.
 - PostgreSQL image contains database binaries, config files, & other dependencies.
 - a Python web app include Python runtime, app code, & dependencies.

Containers

Image Principles

- Images are immutable:
 - Images cannot be modified once created. You can only make a new image or add changes on top of it.
- Container images are composed of layers:
 - Each layer represents a set of file system changes that add, remove, or modify files.

Containers

Finding Images

- Docker Hub is the default marketplace for storing & distributing images.
- A great starter for your own images is:
 - Docker Official Images: a curated set of Docker repositories.
 - Docker Hardened Images: minimal, secure, production-ready images.
 - Docker Verified Publishers: high-quality images from commercial publishers verified by Docker.
 - Docker Sponsored Open Source: images published & maintained by open-source projects sponsored by Docker through its open source program.

Containers

Docker Registry

- a storage & distribution system for Docker images.
- example tools: Harbor, Docker Hub, & JFrog Artifactory.
- setting up a private Docker Registry on port 5000:
- Benefits:
 - centralised location for storing container images.
 - a central location where users can find container images.
 - Docker & Kubernetes integration.

Containers

Docker Registry Best Practices

- **access controls**: Implement proper authentication and authorization to ensure only authorized users can access, push, or pull images from your registry.
- **image versioning**: Use meaningful version tags for your images to facilitate easier rollback and maintenance.
- **clean up**: Regularly audit and clean old images from your registry to save storage and ensure efficiency.
- **automation**: Integrate your Docker Registry with CI/CD pipelines for automated builds, testing, and deployment.

Containers

Registry Vs Repository

- a registry is a centralised location that stores & manages container images.
- a repository is a collection of related container images within a registry.



Containers

Docker Exercises

- Install & run your first container:
 - <https://docs.docker.com/get-started/introduction/get-docker-desktop/>
- Develop with containers:
 - <https://docs.docker.com/get-started/introduction/develop-with-containers/>
- Build & push your first image:
 - <https://docs.docker.com/get-started/introduction/build-and-push-first-image/>

Containers

Container Orchestration

- suppose you want to run more than one container with multiple services, e.g., running databases, message queues, caches, etc.?
- note that containers should do one thing. deployment & management.
- multiple “docker run” commands can be used to start multiple containers but the network will have to be managed, flags needed to connect the containers, etc., apart from the cleanup needed at the end.
- Docker Compose define containers & their configurations in a single YAML file.

Containers

Further Study

- Docker Registry: <https://arnab-k.medium.com/understanding-docker-registry-for-efficient-image-management-def77ba43201>
- Kubernetes Lab: <https://codelabs.developers.google.com/codelabs/cloud-springboot-kubernetes#0>

Containers

References

- Dockerfile: <https://docs.docker.com/reference/dockerfile>
- Docker Registry: <https://www.sysdig.com/learn-cloud-native/what-is-a-docker-registry>
-

Containers

Thank You

Questions?

T: +264 61 207 2054

F: +264 61 207 9054

E: smuchinenyika@nust.na